

# 面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与轻量 DA 成本评估

面向高频 DApp 的 Hybrid TEE-Rollup 可恢复挑战协议与携带证据的 DA 提交机制

作者：杨帆

## 摘要

高频 DApp (Decentralized Application) 如元宇宙状态同步、去中心化社交交互和链上 AI 推理服务，使区块链扩容问题呈现出典型的安全系统权衡：系统既需要较低交互成本和较低延迟，又不能在异常场景下放弃公开可验证性与挑战活性 (protocol liveness)。全链上执行具有较强公开验证属性，但难以承受大量小额、连续状态更新；完全链下执行虽然降低成本，却缺少可公开审计的仲裁路径；单纯依赖可信执行环境 (TEE) 又会引入硬件信任、宿主可用性和远程证明边界。围绕这一矛盾，本文研究 Hybrid TEE-Rollup 中一个范围较窄但关键的问题：当执行者仅向链上提交轻量承诺、完整负载数据 (payload) 位于数据可用性层 (DA)，而挑战 (challenge) 过程又因 timeout 停滞时，协议如何恢复争议、重放 (replay) 证据并完成仲裁结算。本文的目标并非给出生产级 Rollup 完整实现，而是构造并评估三个相互关联的机制接口：可恢复挑战机制 (Recoverable Challenge)、携带证据的紧凑提交 (Evidence-Carrying Compact Commit) 以及 DA 感知成本模型。基于 Python 原型、本地 EVM gasUsed 与 Sepolia 部署记录，实验结果显示 compact commit 平均约为 406 bytes；在给定成本模型与 DA profile 下，compact modular DA sampling 相比 full-onchain calldata 在 payload=8192、prompt length=64、batch size=1000 时得到 96.33% 的配置化估算降本；recoverable challenge 在 timeout 场景下将 challenge success 从 0% 提升至 100%。这些结果用于说明机制接口、活性差异和成本趋势，不应外推为真实 TEE 安全、生产级 DA 或主网费用结论。

**关键词：**Hybrid TEE-Rollup；交互式挑战；数据可用性；高频 DApp；成本摊销；故障恢复

## 1 引言 (Introduction)

高频 DApp 的困难并不只是吞吐量不足。此类应用的单次交互价值通常不高，却要求持续状态更新、低延迟反馈、低链上负担，以及异常场景下的公开可验证仲裁。以链上 AI 推理服务为例，用户连续提交 prompt，执行者在 TEE 中生成 response，并向链上提交 compact commit；完整 prompt、response 与 attestation 保存在 DA 层。正常路径需要尽量轻量，但异常路径必须具备足够的可验证性：当 response 与 DA payload 不一致，或 challenge 在中途 timeout 时，系统不能仅报告“异常已检测”，还需要保证争议能够继续推进到 replay 与 resolve。

现有路线分别缓解了上述矛盾的一部分，但难以同时满足低成本、低延迟、公开可验证和挑战活性四项要求。全链上执行保留了公开验证能力，却将高频 payload、存储和执行成本直接推到链上；完全链下执行降低了延迟和成本，却缺少用户可依赖的公开仲裁路径；纯 TEE 执行可以降低正常路径的验证成本，但系统安全性和可用性仍受硬件、宿主 I/O、远程证明以及服务连续性的影

响。Hybrid TEE-Rollup 的优势在于将 TEE 快速路径与链上异常路径结合起来；相应地，其关键风险也在异常路径：如果缺少恢复机制和证据入口，系统可能在最需要仲裁时失去协议连续性。

timeout 后挑战流程卡死并非普通工程异常，而是协议活性与仲裁连续性（arbitration continuity）问题。传统挑战机制可以关注错误是否能够被发现，但对于高频 DApp 而言，检测只是争议处理的起点：未完成的 challenge 会占用状态、延迟结算，并使错误提交停留在不可结算的中间状态。因此，本文以“故障发生—恢复—重放—仲裁结算”为主线。异常被观察后，协议需要恢复挑战会话；恢复后，compact commit 必须能够指向 replay 所需证据；replay 之后，仲裁结果才能进入 resolve 或 slashing。

轻量提交同样存在安全语义边界。若 compact commit 只是普通 hash，虽然能够压缩链上字节，却无法指明验证者如何取回 payload、验证字段级 membership，或为 replay 构造证据上下文。本文将 compact commit 视为最小可验证证据入口（minimal verifiable evidence entry）：它在正常路径保持链上提交轻量，在异常路径仍能回指 DA payload、Merkle proof、attestation 摘要和执行摘要。实验部分采用合成负载（synthetic workload）、模拟 TEE（simulated TEE）、mock/verifiable DA registry、本地 EVM gasUsed 与 Sepolia 可部署性证据；这些证据用于评估机制接口与趋势，而非声称生产网络性能。

1.1 本文贡献（Contributions）

本文的第一个贡献在于提出可恢复挑战机制（Recoverable Challenge）。在传统挑战流程中，timeout 可能使争议停留在未解决状态，即使系统已经检测到异常，也无法自然推进到 replay 与 resolve。对于高频 DApp，连续结算能力比单次异常检测更为关键。本文将 recover 建模为保持活性的状态转移：该操作不修改争议事实、expected trace 或 claimed trace，而是将停滞的挑战会话恢复到可重放、可仲裁的路径上。

本文的第二个贡献在于设计携带证据的紧凑提交（Evidence-Carrying Compact Commit）。该提交不是普通哈希摘要，而是将 state root、output hash、proof hash、DA pointer 与 DA root 绑定在同一上下文中，使链上对象同时具备 replay-ready、DA-traceable 和 verification-oriented 属性。其意义在于：正常路径仅提交最小摘要，异常路径仍可通过 DA 证据进入验证与重放流程。

本文的第三个贡献在于构建 DA 感知成本模型（DA-aware Cost Model）。该模型不是价格预测器，而是用于分析 payload scaling、batch amortization 与 DA route selection 的轻量化工具。它帮助系统设计者理解 full calldata 在何种条件下会成为结构性瓶颈，批处理摊销在高频 workload 中为何重要，以及 replay-ready DA 证据会如何改变紧凑提交方案的成本边界。

表 1 本文机制推进点与研究作用

| 机制推进                                | 核心作用                     | 论文定位   |
|-------------------------------------|--------------------------|--------|
| 可恢复挑战<br>（Recoverable<br>Challenge） | 在 timeout 或中断后恢复<br>争议流程 | 提升挑战活性 |

| 机制推进                                            | 核心作用                                               | 论文定位           |
|-------------------------------------------------|----------------------------------------------------|----------------|
| 携带证据的紧凑提交<br>(Evidence-Carrying Compact Commit) | 绑定<br>state/output/proof/DA<br>root                | 形成最小证据入口       |
| 故障分类 (Fault Taxonomy)                           | 区分 attestation、DA、<br>timeout、replay 和<br>trace 故障 | 明确异常责任边界       |
| DA 感知成本摊销                                       | 联合 payload、batch 和<br>DA route 分析成本                | 支撑高频 DApp 成本论证 |

表 2 机制语义对比 (Mechanism-Oriented Comparison)

表 2 的作用并非给出能力清单，而是界定本文机制与既有系统的关系：已有系统通常已经具备挑战机制或 TEE 快速路径，本文关注的增量在于 timeout 后的挑战活性、commit 到 DA 证据的绑定关系，以及面向高频 workload 的成本解释。

| 系统        | 挑战机制角色                                | 恢复语义                            | DA 证据绑定                         | 成本模型作用                      |
|-----------|---------------------------------------|---------------------------------|---------------------------------|-----------------------------|
| Optimism  | Fraud proof /<br>dispute game         | 主要依赖挑<br>战窗口假设<br>处理<br>timeout | 非本文比较重<br>点                     | 本文未建模                       |
| Arbitrum  | 交互式争议定位                               | 协议特定的<br>争议推进机<br>制             | 非本文比较重<br>点                     | 本文未建模                       |
| OTR       | TEE 辅助的<br>optimistic<br>verification | timeout 恢<br>复讨论有限              | 部分<br>payload/proof<br>关联       | 本文未建模                       |
| TEERollup | TEE 快速路径与<br>挑战入口                     | 主要作为系<br>统机制讨论                  | 架构层面讨论                          | 未给出轻量 DA 摊销<br>模型           |
| 本文工作      | 可恢复挑战状态<br>机                          | recover 作<br>为保持活性<br>的转移       | commit 到 DA<br>字段 proof 的<br>绑定 | payload/batch/route<br>权衡分析 |

2 背景与威胁模型 (Background and Threat Model)

2.1 Rollup、TEE 与数据可用性 (Data Availability)

Rollup 的基本思想是将大量执行过程移至链下完成，并将状态承诺、证明或争议入口提交到链上。ZK-Rollup 通过有效性证明保证提交状态的正确性，具有较强的验证属性，但证明生成、验证电路设计和复杂系统实现带来较高成本。Optimistic Rollup 假设链下执行结果默认正确，只有

在挑战期内出现争议时才进入 fraud proof 或交互式验证流程，因此常规路径成本较低，但结算延迟和挑战机制复杂度成为关键限制。

TEE attestation 为链下执行可信性提供补充接口。可信执行环境可以在隔离环境中执行程序，并对输入、输出、环境标识和 nonce 生成证明，使外部验证者能够确认某一输出来自指定执行环境。然而，TEE 更适合作为降低正常路径验证成本的组件，而不应替代链上挑战、数据可用性和故障追责机制。

Optimistic challenge 是 Rollup 系统中处理异常的重要机制。典型流程包括提交状态承诺、开启挑战、交互式缩小争议范围，并在必要时执行单步重放或链上仲裁。对于长执行轨迹，二分定位能够将争议定位复杂度从线性扫描降低到对数轮次；但 challenge 本身也可能遭遇 timeout、应答缺失或证据不可用，因此其活性和恢复性需要被纳入协议设计。

数据可用性 (Data Availability, DA) 关注链下或链外数据是否能够被需要的验证者获取。模块化区块链和 LazyLedger 等路线强调将数据可用性从执行验证中解耦，使共识层主要负责数据排序和可用性保证。对于 Hybrid TEE-Rollup 而言，DA 不仅用于降低存储成本，还需要支撑后续 challenge 的证据回溯。因此，compact commit 必须与 DA proof 建立可验证联系。

## 2.2 问题范围与对抗假设 (Problem Scope and Adversarial Assumptions)

在上述背景下，本文进一步限定研究问题：Hybrid TEE-Rollup 如何在保持正常路径低成本的同时，保留公开、可恢复且有证据支撑的异常路径。成本维度要求链上提交不随完整 payload 线性增长；可验证性维度要求 compact commit 能够指向可检查的 DA 证据；活性维度要求 timeout 不会使仲裁流程永久停滞。

攻击面覆盖提交者、执行环境、DA 绑定和挑战流程。本文假设执行者或提交者可能提交错误 response、篡改 DA payload、提供无效 attestation，或在挑战流程中延迟响应；DA 层可能出现 payload 缺失、Merkle proof 损坏或 root 不一致；挑战流程本身也可能因超时、中断或 trace 长度不一致而无法自然进入 resolve 阶段。验证者、挑战者或 watchdog 可以观察链上提交和 DA 证据，并在发现异常时发起或恢复挑战。

范围说明。本文的威胁模型不覆盖真实硬件侧信道、远程证明供应链故障、真实 DA 网络的经济安全、主网 MEV 以及拥堵导致的费用波动。该范围限定并不构成生产级声明，而是说明当前原型证据能够支撑的内容：协议状态连续性、证据绑定关系、配置化成本趋势，以及合约层面的对应关系。

## 3 系统概览 (System Overview)

本节说明上述机制如何组成端到端系统。系统包含三个角色：执行层负责产生正常路径下的 response 与 attestation；DA 层负责保存 replay 所需证据；验证层负责将故障转化为 challenge、recover、replay 与 resolve 等状态转移。运行示例仍以高频 AI 推理 workload 为

主：大量低价值请求不应迫使完整 payload 上链，但一旦发生争议，系统仍需保留公开仲裁所需的证据。

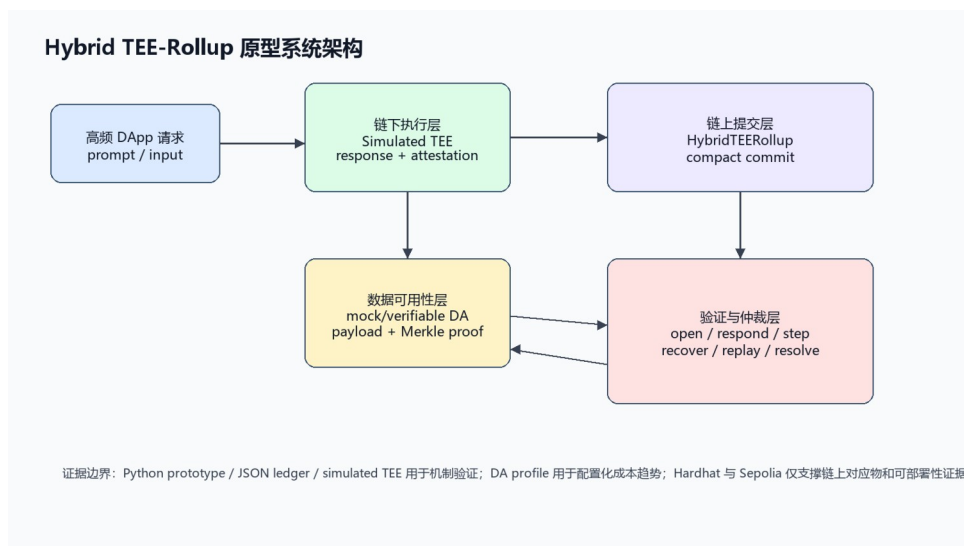


图 1 Hybrid TEE-Rollup 原型系统架构

如图 1 所示，系统由高频 DApp 请求入口、链下执行层、数据可用性层、链上提交层和验证仲裁层组成。正常路径中，执行层输出 response 与 attestation，数据层保存完整证据，链上提交层仅保存 compact commit。异常路径中，验证仲裁层通过 DA pointer 取回证据，并以 challenge、recover、replay 和 resolve 推进状态。这个划分使低成本正常路径与可仲裁异常路径保持同一证据上下文。

### 3.1 执行层与正常路径（Execution Layer and Normal Path）

执行层负责处理高频 DApp 请求。当前原型采用 deterministic mock model 或可选模型后端生成响应，并使用 simulated TEE attestation 对输入哈希、输出哈希、nonce 和执行环境标识进行绑定。attestation 的作用是为链下执行结果提供最小可信摘要，使后续验证层能够检查提交结果是否与声明的执行过程一致。

在本文原型中，simulated TEE 的作用是建立“链下执行摘要—链上提交—异常挑战”的接口关系。真实 TEE 安全性不在本节展开，而在“讨论与局限性”中集中说明。

### 3.2 数据可用性接口（Data Availability Interface）

DA 接口是 compact commit 具有安全意义的关键位置。数据层保存完整 payload 及其可验证结构，包括 prompt、response、attestation、payload hash、leaf hashes、Merkle root 和字节级 Merkle proof；链上仅保存 compact commit。由此，字节压缩与证据保留被明确区分：正常路径避免完整 payload 上链，异常路径仍可恢复 replay 所需的 payload 上下文。

在运行示例中，如果执行者提交的 response 被替换，验证者可以从 DA entry 中取回 response 字段及其 Merkle proof，并与链上 da\_merkle\_root 检查一致性；如果 payload 缺失，则异常应被归类为 DA availability fault，而不应被误写为已完成 replay 的执行错误。

### 3.3 验证层与恢复触发 (Verification Layer and Recovery Trigger)

验证层负责处理异常路径。当前原型实现了 challenge open、respond、step、recover、replay 和 resolve 等状态推进。其目标不是将所有执行都搬到链上，而是在出现争议时，通过证据加载、二分定位、单步重放和仲裁结算，将异常状态推进为可解释的结果。

验证层同时承担故障分类 (fault taxonomy) 职责。本文将故障划分为证明故障 (attestation fault)、DA 故障、timeout 故障、replay 故障和轨迹不一致 (trace inconsistency)。不同故障对应不同处理路径：DA unavailable 表示数据层无法提供 payload；DA proof invalid 表示 payload 与链上 root 不一致；trace length mismatch 表示执行轨迹与声明结构不一致；response tampering 则可能同时导致 payload hash、attestation 和 state root mismatch。

在 Solidity 映射中，原型包含 MockTEEverifier、MockDARegistry、MerkleVerifier 和 HybridTEERollup 等合约，用于表达链上状态推进和最小验证逻辑。本地 EVM 合约并非 Python 原型的逐行移植，而是抽取适合链上表达的提交、验证、挑战、恢复和结算路径。

## 4 可恢复挑战协议 (Recoverable Challenge Protocol)

### 4.1 协议模型与形式化定义 (Protocol Model and Definitions)

本节给出恢复机制成立所需的协议边界。recover 操作只有在恢复流程推进而不修改争议事实时才具有安全意义。下面的定义将会话状态与 Challenge Facts 区分开，使 timeout 后的恢复可以被理解为活性层转移，而不是隐藏的正确性裁决。

#### 定义 1 (Challenge Session, 挑战会话)

挑战会话定义为： $cs = (id, s, F, T)$ 。

其中，id 表示会话标识；s 表示当前状态；F 表示 Challenge Facts（挑战事实集合）；T 表示超时参数。

#### 定义 2 (Challenge State Machine, 挑战状态机)

挑战状态机定义为： $M = (S, E, \delta)$ 。

其中，S 为状态集合；E 为事件集合； $\delta: S \times E \rightarrow S$  为状态转移函数。

#### 定义 3 (Recovery Function, 恢复函数)

恢复函数定义为： $Recover(cs) = cs'$ 。

恢复仅改变活性相关状态，不改变 Challenge Facts，即满足： $F(cs) = F(Recover(cs))$ 。

## 协议性质 (Protocol Property)

基于上述定义，recover 是活性层变换，而非正确性裁决。Recover 允许会话从 TIMED\_OUT 等停滞状态回到可推进状态，但不重写 commit、DA root、payload hash、trace 或 mismatch evidence。该边界将恢复语义与 slashing 语义区分开，并说明恢复操作不会改变 correctness；后续 replay 仍基于原始 Challenge Facts。

## 4.2 超时、恢复、重放与仲裁结算 (Timeout, Recovery, Replay, and Resolution)

可恢复挑战协议 (Recoverable Challenge Protocol) 的目标，是将挑战从一次性错误检测流程扩展为异常场景下可恢复推进的协议状态机。其基本状态包括 PENDING、OPEN、RESPONDED、NARROWING、READY\_FOR\_REPLAY、REPLAYED、RESOLVED、SLASHED 和 FINALIZED。

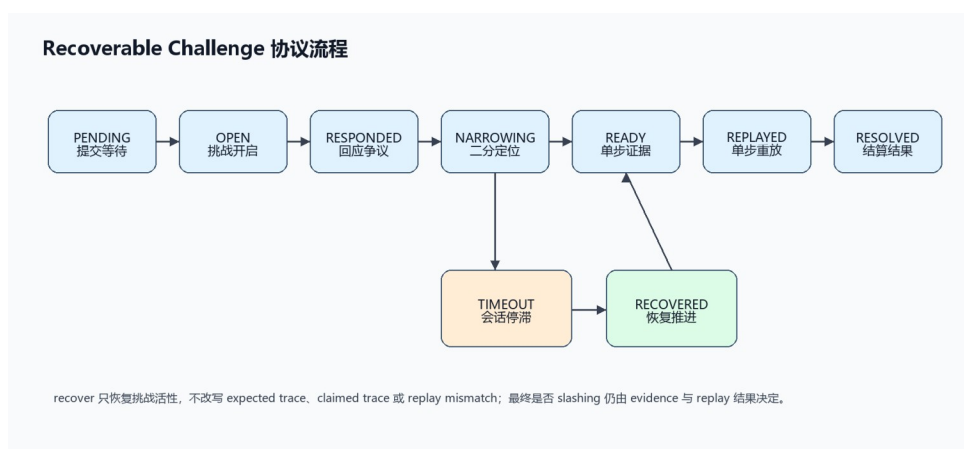


图 2 可恢复挑战协议流程 (Recoverable Challenge Protocol)

当交易处于 PENDING 状态时，挑战者可以调用 open 进入 OPEN 状态。系统加载 compact commit、DA status 和相关 mismatch evidence。随后，被挑战方进入 respond 阶段，提供其对争议的回应。若争议涉及执行轨迹，协议进入 step 阶段，通过二分缩小 (bisection narrowing) 定位争议区间。对于长度为  $n$  的 trace，理想情况下定位轮次接近  $\log_2(n)$ ，从而避免对长执行轨迹进行线性扫描。

当争议区间被缩小到单步时，协议进入单步重放 (single-step replay)。replay 阶段比较 expected hash 与 claimed hash，判断局部执行是否一致。如果重放失败，resolve 阶段可以将交易标记为 SLASHED；如果重放通过，则挑战不成立或进入相应结算逻辑。

recover 是协议中的关键状态转移，因为它在不改变证据内容的前提下修复流程推进。在不可恢复的 challenge 中，timeout 可能使会话停留在“争议已知但无法重放和结算”的状态。可恢复挑战机制引入 watchdog/operator 恢复路径，使超时会话重新回到可 step、replay 和 resolve 的状态。这里需要区分活性与正确性：recover 恢复的是前者，而后续 replay 仍负责验证后者。

协议流程可分为三条路径。第一是正常路径：若挑战期内没有有效争议，交易最终进入 FINALIZED。第二是可仲裁异常路径：若 response tampering 或 trace length mismatch 等执行相关故障出现，协议通过 open、respond、step、replay 与 resolve 完成争议定位和结算。第三是 timeout 恢复路径：若挑战过程因超时中断，watchdog 或 operator 可以调用 recover 将会话推进到 RECOVERED，并继续后续 replay 与 resolve。该划分有助于避免将“异常检测”“活性恢复”和“经济惩罚”混写为同一性质。

### 4.3 正确性边界与协议性质 (Correctness Boundary and Protocol Properties)

性质一，DA proof 绑定性质。compact commit 中的 da\_merkle\_root 与 DA entry 中的字段级 Merkle proof 绑定，因此 prompt、response 或 attestation 字段被替换后，会导致 payload hash、leaf hash 或 Merkle proof 与链上 root 不一致。该性质不依赖真实 DA 网络，只说明当前 verifiable DA 结构能够检测字段篡改。

性质二，recover 的活性性质。recover 操作的目标是恢复超时挑战会话，使其继续进入 step、replay 和 resolve；它不修改 expected trace、claimed trace 或 replay mismatch 的事实。因此，recover 改变的是挑战流程的活性连续性，而非争议正确性本身。

性质三，二分定位性质。对于长度为  $n$  的执行 trace，challenge-step 每轮缩小争议区间，理想轮次接近  $\log_2(n)$ 。本文实验中 trace steps 为 4、8、16、32、64、128 时，平均二分轮次分别为 2、3、4、5、6、7，与理论对数趋势一致。

## 5 携带证据的紧凑提交与 DA 绑定 (Evidence-Carrying Compact Commit and DA Binding)

本节说明 compact commit 为什么不是单纯压缩结构。其研究价值在于携带异常路径所需的最小证据关联。该提交绑定 state\_root、output\_hash、proof\_hash、da\_pointer 和 da\_merkle\_root：state\_root 约束输入、输出与环境摘要；output\_hash 约束 response；proof\_hash 约束 attestation；da\_pointer 定位完整 payload；da\_merkle\_root 则承诺字段级 proof membership。

这种绑定关系使验证者能够从链上提交进入 DA 证据结构。若 response 被篡改，验证者检查 response leaf 与 Merkle root；若 attestation 与输出不一致，proof\_hash 和 payload hash 形成交叉约束；若 DA proof 损坏，系统可以将异常定位为 DA proof invalid，而不是混同为执行重放失败。由此，compact commit 支持后续 challenge/replay 的证据加载，而不仅服务于链上字节压缩。

从对抗角度看，提交者可能替换 payload 字段、隐藏 DA entry、提交与 attestation 不一致的 output，或在争议过程中延迟响应。compact commitment 本身并不能解决真实 DA 网络可用性 or 真实 TEE 信任问题；其更窄的贡献在于明确链上最小承诺需要绑定哪些证据字段，才能使后续 challenge 具备 replay-ready 能力，而不仅是节省链上字节。

## 6 DA 感知成本模型 (DA-aware Cost Model)

成本模型关注 full-onchain 与 compact commit + DA 在高频场景下的结构性差异。full-onchain 路线将完整 payload 作为链上数据提交，其成本随 payload size 增大近似线性增长。compact commit 路线只将固定摘要结构提交到链上，将完整 payload 放入 DA 层，并通过 DA root 和 Merkle proof 保留验证能力。

模型比较四类 DA 路径：full-onchain calldata、compact external DA、compact EIP-4844-like 和 compact modular DA sampling。该比较属于配置化估算，应被理解为系统权衡模型，而非真实世界价格。payload size 决定完整数据发布何时成为主要开销；batch size 决定固定提交开销能否被摊薄；DA route 则决定将 replay 证据移出 calldata 后引入的边际成本和依赖关系。

模型表达为  $C_{total} = C_{commit} + C_{DA} + p_{challenge} * C_{challenge}$ 。其中  $C_{commit}$  表示链上 compact commit 或 full payload 提交成本， $C_{DA}$  表示不同 DA route 下的数据成本， $C_{challenge}$  表示发生争议时的挑战、恢复、重放和结算成本， $p_{challenge}$  表示争议发生概率。批处理后单笔摊销成本为  $C_{amortized} = C_{total} / batch\_size$ 。当前实验主要比较不同 route 在相同参数下的结构性趋势，而不估计真实业务中的  $p_{challenge}$ 。

从系统角度看，batch size 越大，固定提交成本越容易被摊薄；payload 越大，full-onchain calldata 的线性成本越突出；DA route 越接近低边际数据成本，compact commit + DA 的收益越明显。当前 Python prototype 中的成本实验属于配置化估算模型，本地 EVM gasUsed 则用于补充链上状态推进基线，两者在实验评估中需要明确区分。

## 7 实验评估 (Evaluation)

实验评估不仅验证原型能否运行，还围绕结果含义组织对比分析。RQ1 关注 compact commit 是否在保留证据关联的同时隔离 payload 增长；RQ2 关注 batch amortization 与 DA route selection 如何改变高频 workload 的可行性边界；RQ3 关注恢复机制能否将 timeout detection 推进为仲裁完成；RQ4 区分哪些 fault 仅可检测，哪些可以 replay 或 slashing；RQ5 则检验协议是否具有合约层对应物。所有 workload 均为 synthetic；成本数值属于配置化估算或本地 EVM gasUsed 基线；Sepolia 证据仅表示可部署性，不代表生产性能。

本文成本实验使用如下抽象模型：

$$C_{total} = C_{commit} + C_{DA} + p_{challenge} * C_{challenge}$$

$$C_{amortized} = C_{total} / batch\_size$$

其中， $C_{commit}$  表示配置中的链上提交成本， $C_{DA}$  表示配置化数据发布成本， $C_{challenge}$  表示 challenge、recover、replay 与 resolve 的异常路径成本， $p_{challenge}$  表示争议发生概率。实验并不估计真实业务中的  $p_{challenge}$ 、blob 市场价格、DA 网络费用或生产网络交易费用，而是用于比较受控 synthetic 输入下的结构性趋势。

## 7.1 RQ1：紧凑提交能否降低链上提交规模

RQ1 隔离考察提交结构。Python 原型在模拟 TEE 下生成合成账本条目（synthetic ledger entries），并比较 full payload 与 compact commit 的 canonical JSON 字节数。这里的目的是不是预测 calldata 价格，而是检验 payload 增长是否被从链上对象中隔离，同时 DA root 和 proof pointer 是否仍能保留 replay 路径。

度量指标包括 full\_bytes、compact\_bytes 和  $\text{byte\_reduction\_ratio} = (\text{full\_bytes} - \text{compact\_bytes}) / \text{full\_bytes}$ 。full\_bytes 表示完整 prompt、response、attestation 及相关字段都作为 payload 提交时的字节规模；compact\_bytes 则只统计 state\_root、output\_hash、proof\_hash、da\_pointer 与 da\_merkle\_root 等摘要字段。

对比结果清楚显示了这种隔离关系：compact commit 基本保持在 406 bytes 左右，而 full payload bytes 随 payload size 增大而增长。当 payload size 为 8192 时，prompt length 为 64 / 256 的 full payload bytes 分别约为 50068.84 / 50262.84，而 compact bytes 仍由提交 schema 固定。该结果说明，应用 payload 不再由链上对象直接承载，但 DA pointer 与 Merkle root 仍保留了异常路径所需的证据关联。

这一结果的意义是有限但关键的：compact commit 将 payload scaling 与链上提交规模分离，同时不丢失 replay readiness。仅保存非结构化 hash 的方案同样可以很小，但无法提供指向 prompt、response、attestation 和字段级 proof evidence 的 DA-traceable 路径。

这一结论的解释边界是编码层面的：字节数来自 Python prototype 的 canonical JSON 表达，不能直接等价于真实链上 calldata、storage layout 或 ABI 编码后的完整成本；真实 attestation 与 DA 证明大小也会改变绝对数值。

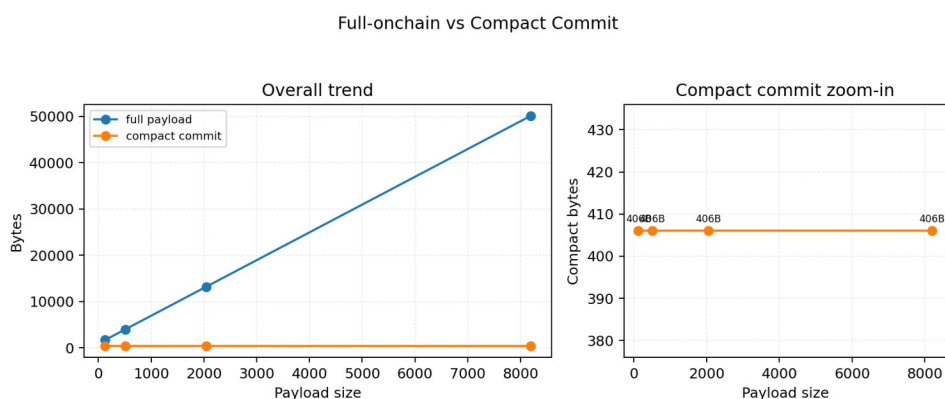


图 3 full payload 与 compact commit 的字节规模对比：payload 增长被隔离到 DA 侧，而 compact commit 仍保留 replay 所需的证据入口

## 7.2 RQ2：批大小如何影响 DA 感知摊销成本

RQ2 关注提交策略在何种条件下会改变高频合成负载的系统可行性。实验比较 full\_onchain\_calldata、compact\_external\_da、compact\_eip4844\_like 和

compact\_modular\_da\_sampling 四类路线，参数覆盖 payload size、prompt length 和 batch size。文中报告的 amortized gas 均来自脚本 profile 的配置化估算，不代表真实网络费用。

主要指标为  $\text{amortized\_gas} = \text{total\_gas} / \text{batch\_size}$  和  $\text{reduction\_ratio} = (\text{full\_onchain\_gas} - \text{target\_gas}) / \text{full\_onchain\_gas}$ 。total\_gas 来自配置化 profile，而不是主网交易回执；因此这里讨论的是结构性趋势，而不是实时网络价格。

趋势表明，当 payload publication 成为主要开销时，batch amortization 的作用会更加明显。以 payload=8192、prompt length=64 为例，full\_onchain\_calldata 的估算 gas 从 batch size=1 时的 801101.44 下降到 batch size=1000 时的 801.10；compact\_modular\_da\_sampling 从 29026.98 下降到 29.03。batch size=1000 时得到的 96.33% 配置化降本，反映的是在大 payload 与高 batch 摊销条件下，将 replay 证据移动到低边际成本 DA profile 的结构性优势，而不是主网费用结论。

系统层面的观察是，payload scaling 与 batch size 共同影响成本边界。当 payload 成为主要部分时，full calldata 会因每个更大 response 都增加链上字节而迅速劣化；当大量交互可以共享固定提交开销时，compact DA 路线更具吸引力。但这种收益同时引入 DA 依赖和 proof retrieval 假设。因此，高频 DApp 设计者需要评估的是结构性权衡，而不是在单一配置下简单判断哪条路线“更便宜”。

96.33% 是给定 cost model 与 DA profile 参数下的趋势结果。它说明 compact modular DA sampling 在大 payload 与大 batch 条件下具有结构性优势，但真实公网费用仍会受到 calldata 定价、blob 价格、交易拥堵、DA 网络实现和业务争议率影响。

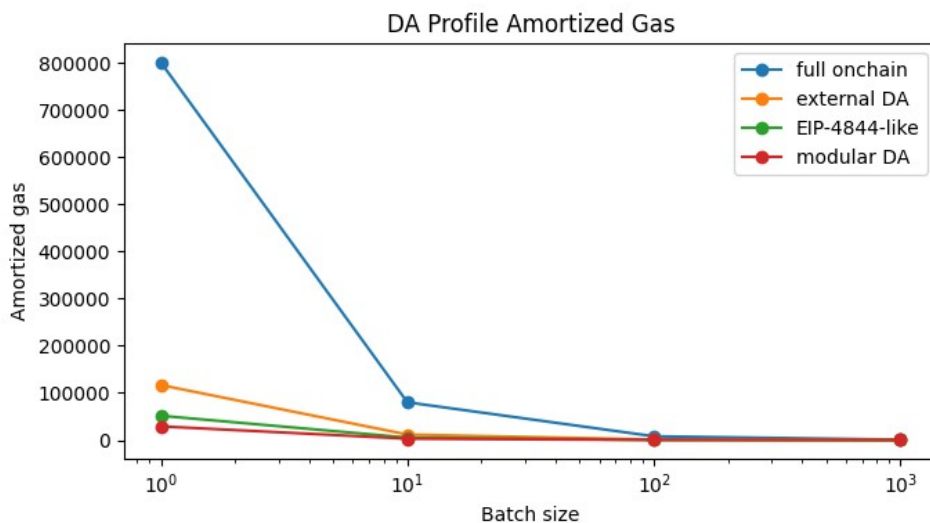


图 4 不同 DA profile 下的摊销 gas 趋势：当 payload publication 成为主要开销时，batch amortization 与 DA route selection 共同决定结构性成本边界

## 7.3 RQ3 : 恢复机制能否改善 timeout 场景下的挑战活性

RQ3 检验本文的核心活性主张。Python 原型在 no-recover 与 recoverable 两条路径中使用相同的合成挑战状态机, trace steps 覆盖 4、8、16、32、64、128, challenge timeout 设为 2 秒, max bisection rounds 设为 16。关键问题是: timeout 被检测后, 会话是否仍能继续进入 replay、resolve 与 slashing。

这里记录 avg\_bisection\_rounds、avg\_expected\_log2\_rounds、recovery\_success\_rate、challenge\_success\_rate 和 slashed\_rate。avg\_bisection\_rounds 用来检查争议定位复杂度是否接近理论预期, challenge\_success\_rate 与 slashed\_rate 则反映仲裁是否真正走到完成状态。

二分定位行为与理论对数趋势一致, 但更关键的是 no-recover 与 recover 的对比。在 challenge\_timeout\_no\_recover 中, detection rate 达到 100%, 但 challenge success 和 slashing 均为 0%; 也就是说, 会话已经被识别为异常, 却未完成仲裁。在 challenge\_timeout\_recover 中, detection、recovery、challenge success 和 slashing 在 synthetic 场景下均达到 100%。该对比说明, timeout detection 并不等价于协议推进。

这一结果说明, recover 改变的是流程推进连续性, 而非争议正确性本身。它不重写 expected trace、claimed trace 或 replay mismatch, 而是重新打开这些事实被检验和结算的路径。对于高频 DApp 而言, 大量小额争议若长期停留在未结算状态, 即使每个 timeout 都可被检测, 也会形成系统层面的可用性问题。

当前实验中的 challenge trace、expected hash、claimed hash 和部分 replay evidence 由原型逻辑或链下流程提供。因此, RQ3 支撑的是 recoverable state machine 的活性主张, 而不是完整链上 fraud proof、FPVM 或真实 ML/TEE 执行验证的安全性。

### 7.3.1 恢复开销分析 (Recovery Overhead Analysis)

利用表 4 (原表 3) 中已有本地 EVM 数据, challenge step 的平均 gasUsed 为 93807, challenge recover 为 156891。由此可得恢复相对单轮 challenge step 的开销比约为: Recovery Overhead =  $156891 / 93807 \approx 1.67$ 。

在本地 EVM 基线中, recover 的 gasUsed 高于单轮 challenge step, 但该成本发生在异常路径。真正相关的对比不是正常路径吞吐与 recover 开销, 而是“未解决仲裁”与“有界恢复转移”之间的差异。Hardhat 本地测量得到的 1.67x 比例仅描述合约路径的相对 gasUsed, 不代表主网定价或生产费用行为。

上述数值来自 Hardhat 本地 EVM 原型测量, 用于解释异常路径的相对成本; 主网 gas 价格、拥堵条件和生产级合约优化不在该测量范围内。

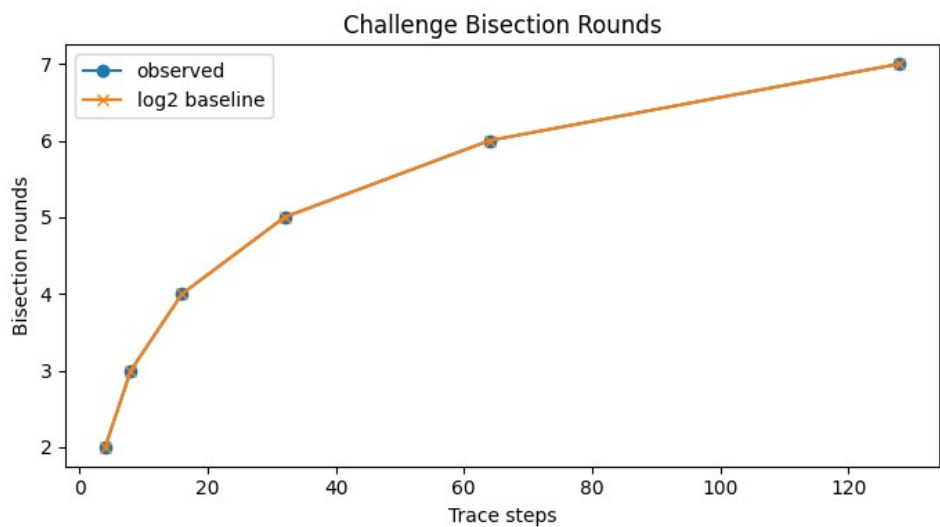


图 5 二分挑战轮次与 trace steps 的关系

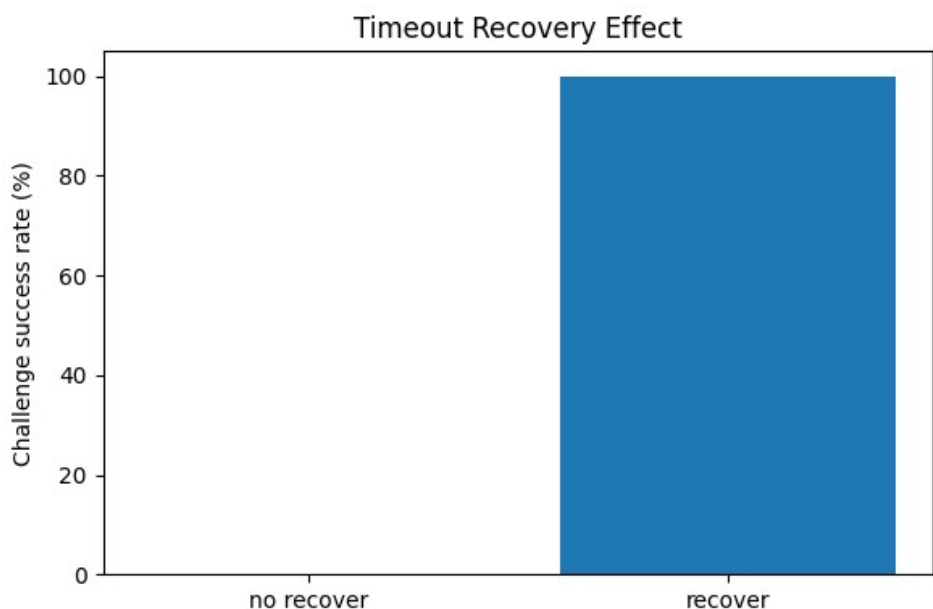


图 6 timeout recover 对挑战完成率的影响：no-recover 可以检测 timeout，但 recoverable challenge 才能将争议推进到 replay/resolve

#### 7.4 RQ4：不同故障场景能否被检测、恢复或惩罚

RQ4 采用面向故障分类的安全评估方式。实验注入八类 synthetic scenario，并区分其在当前原型中是否可检测、可恢复、可重放或可惩罚。这样可以避免一种常见的过度概括：不同 fault 不应仅因为能够被检测，就被强行映射为相同的 slashing 语义。

指标为 detection\_rate、challenge\_success\_rate 和 slashed\_rate。为了避免把语义不同的异常混在一起，实验分析分为四类：正常路径、可检测但未统一惩罚、挑战与惩罚闭环，以及 timeout 活性对比。

对比结果区分了四类结果。Normal path 样本不会触发误惩罚；attestation invalid、DA unavailable 和 DA proof invalid 能够被检测，但当前原型尚未将其统一映射到 slashing；response tampering 与 trace length mismatch 由于存在可比较的执行轨迹，可以进入 challenge、replay 与 slashing 路径；timeout 只有在 recoverable 路径中才能继续推进到可惩罚状态。该结果说明，检测、恢复、重放和惩罚是不同的协议能力。

RQ4 的重点不是宣称所有 fault 已经闭环，而是明确当前闭环在哪里。响应篡改、trace 不一致和 timeout recovery 已形成较完整的 challenge + slashing 路径；attestation fault 与 DA fault 则主要体现为稳定检测、拒绝或报告能力。该区分使 evaluation 更接近安全论文中的 fault-specific reasoning，而不是把“能发现异常”误写成“都能惩罚异常”。

后续仍需细化 fault taxonomy。特别是 DA unavailable、DA proof invalid 和 attestation invalid，应进一步区分何时直接拒绝、何时触发恢复、何时进入链上仲裁，以及何时应施加经济惩罚。

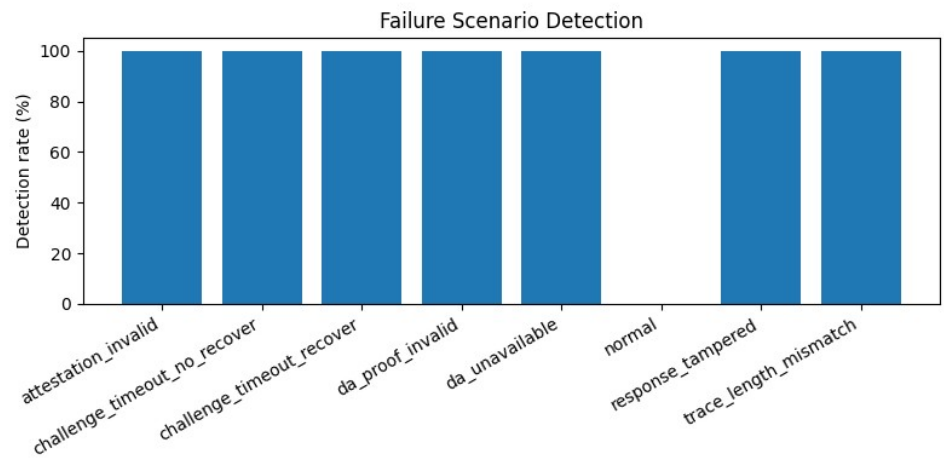


图 7 失败场景检测与仲裁结果：面向故障分类的视角区分 detectable、recoverable、replayable 与 slashable，而不是将所有 fault 统一为同一种惩罚语义

表 3 失败场景分类与当前证据边界

| 类别        | 场景                                                    | 当前结果                                     | 证据边界              |
|-----------|-------------------------------------------------------|------------------------------------------|-------------------|
| 正常路径      | normal                                                | 不触发检测或 slashing                          | 说明正常路径不误报         |
| 可检测但未统一惩罚 | attestation_invalid, da_unavailable, da_proof_invalid | 检测率 100%, challenge success/slashed 为 0% | 说明可检测或拒绝，不说明已统一惩罚 |

| 类别           | 场景                                                               | 当前结果                                               | 证据边界                  |
|--------------|------------------------------------------------------------------|----------------------------------------------------|-----------------------|
| 挑战 + 惩罚闭环    | response_tampered,<br>trace_length_mismatch                      | detection/challenge<br>success/slashed 均<br>为 100% | 说明典型执行异常可进入<br>闭环     |
| timeout 活性对比 | challenge_timeout_no_rec<br>over vs<br>challenge_timeout_recover | no recover 为 0%,<br>recover 后为 100%                | 说明 recover 改善仲裁<br>活性 |

## 7.5 RQ5 : Solidity、本地 EVM 与 Sepolia 证据能否支撑链上对应物

RQ5 关注协议设计是否具有具体的合约层对应物。Solidity 原型和 Hardhat 本地链测量了 register DA、submit rollup、challenge open/respond/step/recover/replay/resolve 与 finalize 等关键 gasUsed 路径；Sepolia 部署记录则提供 chain id、部署区块、合约地址和 ABI/address index。这些证据支持协议到合约的映射关系和可部署性，而非生产可用性。

本地 EVM 指标是各关键函数调用的 gasUsed。Sepolia 证据包括 chain id、部署区块、合约地址和 ABI/address index。前者用于观察链上状态推进路径的基线开销，后者用于说明同一组核心合约已经具备公开测试网 deployability evidence。

本地 EVM 平均 gasUsed 包括：register DA 113526.00, submit rollup 299033.67, challenge open 283769.00, challenge respond 65886.00, challenge step 93807.00, challenge recover 156891.00, challenge replay 135632.00, challenge resolve 113402.00, finalize 33477.00。Sepolia 部署记录则给出了公开测试网链上地址与 ABI/address index。将这两类证据放在一起看，可以说明 Python prototype 中的提交、挑战、恢复、重放和结算路径已经被抽取成 Solidity 状态推进操作，但还不能说明它们已经经历生产网络压力测试。

这一结果支持的是链上对应关系主张。Python 状态机中的操作并非仅停留在伪代码层面，而是被映射为 Solidity 函数，并通过本地 EVM gasUsed 基线和公开测试网部署记录得到支撑。该主张仍然是有限的：它说明可部署性与合约路径对应关系，不说明主网性能、经济安全或审计就绪性。

本地 EVM gasUsed 不能等价于公网费用，Sepolia 部署也不能说明生产可用性、主网性能或经济安全。当前 Sepolia 证据仅支持 deployability evidence。

表 4 本地 EVM 关键路径平均 gasUsed：这些数值是 local EVM 合约路径基线，不代表生产网络费用

| 操作                | 平均 gasUsed | 解释                                       |
|-------------------|------------|------------------------------------------|
| register DA       | 113526.00  | MockDARegistry 记录 payload hash 与 DA root |
| submit rollup     | 299033.67  | compact commit 提交路径                      |
| challenge open    | 283769.00  | 打开争议并加载证据                                |
| challenge respond | 65886.00   | 被挑战方响应                                   |

| 操作                | 平均 gasUsed | 解释        |
|-------------------|------------|-----------|
| challenge step    | 93807.00   | 二分定位的单轮推进 |
| challenge recover | 156891.00  | 恢复超时挑战会话  |
| challenge replay  | 135632.00  | 单步重放仲裁    |
| challenge resolve | 113402.00  | 结算争议结果    |
| finalize          | 33477.00   | 挑战期后最终确认  |

表 5 Sepolia 部署信息：该证据支持可部署性与协议到合约的映射关系，不支持生产性能主张

| 项目                 | 数值                                         |
|--------------------|--------------------------------------------|
| Network / Chain ID | Sepolia / 11155111                         |
| 部署区块               | 10825904                                   |
| MockTEEVerifier    | 0xC42CB0Cf0D112Bd59E0f212F2DB2002ca50a8b6A |
| MockDARegistry     | 0xc42C64a7De05bf3ad4f1c75CbD8E0506F7De99Ff |
| HybridTEERollup    | 0xB9B72f10bB8aBC1ed090f1B0443Fe57189c497Fd |

## 7.6 可复现性与证据类型（Reproducibility and Evidence Types）

Python 原型位于 `project_code/tee_rollup_demo`，正式实验脚本为 `project_code/scripts/run_paper_experiments.py`，输出 `cost_grid.csv`、`challenge_grid.csv`、`failure_scenarios.csv`、`summary.json` 和 `figures` 目录。本地 EVM gas 脚本为 `project_code/evm/scripts/measure_gas.js`，输出 `paper_outputs/evm_gas/measured_gas_report.md` 与 `measured_gas_summary.json`。Sepolia 部署记录位于 `project_code/evm/deployments/sepolia.json`。本次版本还保留 `comparison_figures` 目录和 `comparison_experiments.md`，用于记录 baseline comparison、fault-oriented comparison 与 DA route tradeoff 的派生图表和说明。

复现实验可使用以下命令：`python -m unittest discover -s tests -v`；`python scripts/run_paper_experiments.py --clean --samples 100 --payload-sizes 128,512,2048,8192 --batch-sizes 1,10,100,1000 --trace-steps 4,8,16,32,64,128`；在 `project_code/evm` 目录下运行 `npm.cmd test` 和 `npm.cmd run measure:gas`。上述命令分别对应机制验证、配置化成本趋势和本地 EVM 状态推进基线，不能相互替代。

## 7.7 对比型实验评估（Comparison-Oriented Evaluation）

前述 RQ 验证了机制本身，本小节进一步说明对比实验的意义。对于本文而言，仅证明原型能够运行并不足够，还需要说明某一机制相对于合理 baseline 改变了什么结果。因此，新增对比实验分别回答：没有 recover 会怎样、没有结构化证据绑定会怎样、不同故障类别如何被处理，以及不同 DA 路径假设下成本边界如何变化。

7.7.1 recover 与 no-recover 的活性对比

基线设置如下。no-recover baseline 使用相同的 timeout workload，但在挑战会话超时后不调用 watchdog recovery；recoverable challenge 则在 timeout 后调用 recover，并继续进入 replay 与 resolve。两者使用相同的合成 trace 构造、timeout 配置和原型状态机，因此可以隔离 recover 对协议活性的影响。

| 场景                    | 检测   | 重放   | 结算   | Slashing | 终止状态 | 平均恢复延迟                    |
|-----------------------|------|------|------|----------|------|---------------------------|
| no-recover            | 100% | 0%   | 0%   | 0%       | 0%   | N/A                       |
| recoverable challenge | 100% | 100% | 100% | 100%     | 100% | timeout 阈值后 1.0s<br>(配置值) |

表 6 synthetic timeout workload 下 recover 与 no-recover 的对比

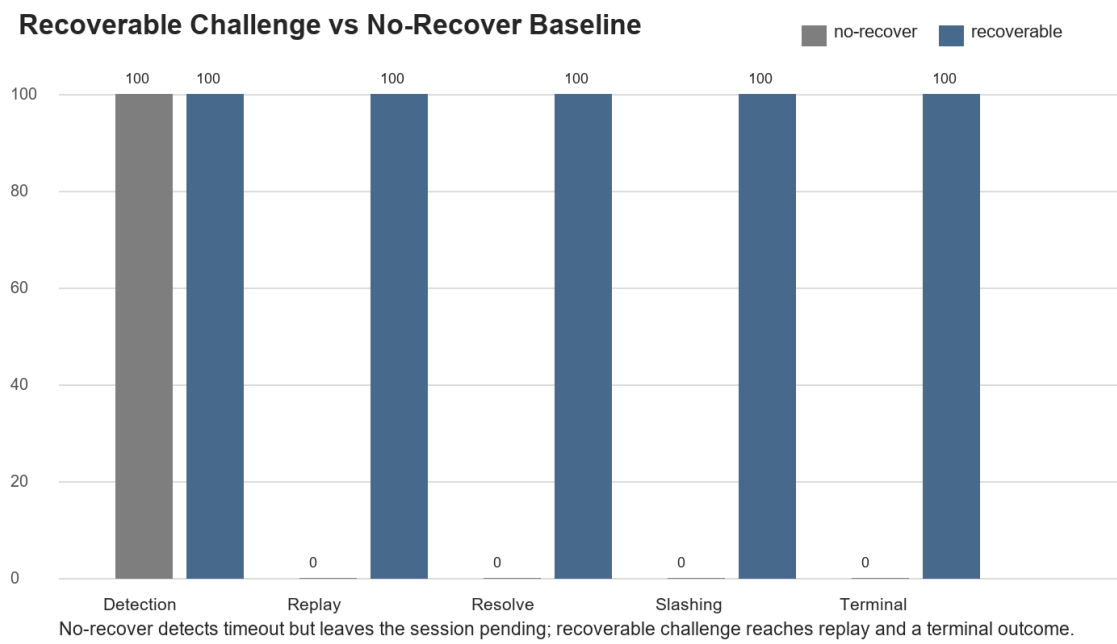


图 8 recover 与 no-recover 在检测、重放、结算、slashing 和终止状态上的对比

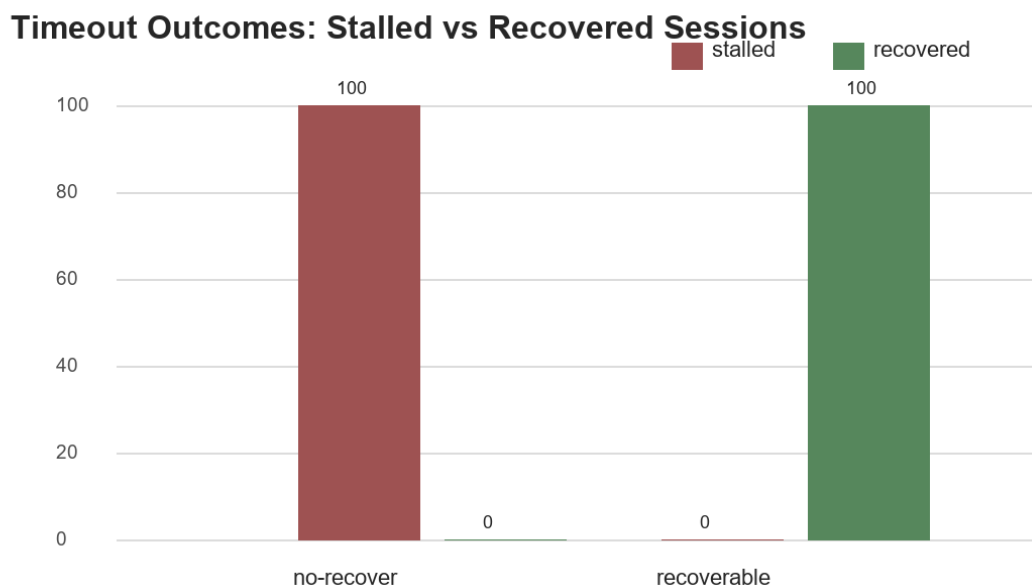


图 9 timeout 会话在无恢复与有恢复条件下的停滞/恢复结果对比

### Replay/Recovery Timeline under Timeout

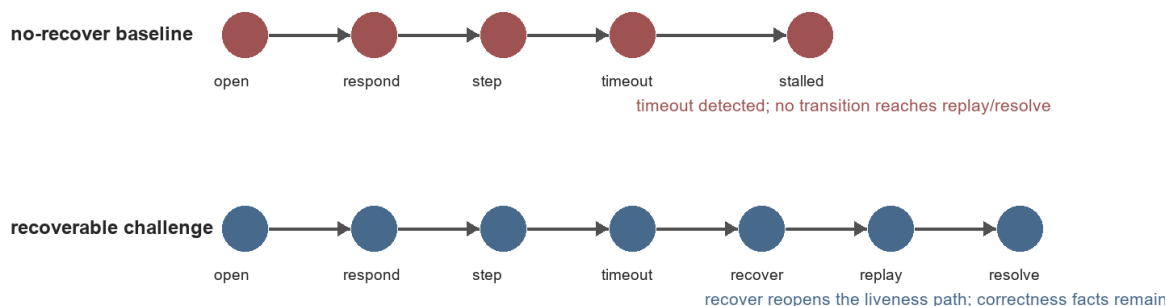


图 10 恢复与重放流程时间线: recover 改变协议活性, 而不改变争议事实

可以观察到, 两条路径都能够检测 timeout, 但只有 recoverable 路径能够继续进入 replay 与 resolve。对比结果表明, detection\_rate 不能单独作为该协议的充分安全指标: 如果缺少恢复转移, timeout detection 仍会使挑战会话停留在 pending 状态。因此, recover 的作用应理解为保持活性的状态转移, 而不是提高判断准确率或绕过正确性验证。

### 7.7.2 紧凑提交与完整负载的证据保留成本对比

compact commit 与 full payload 的对比使用与 RQ1 相同的合成负载。full payload 方案将 prompt、response、attestation 等字段放入链上 payload；compact commit 则仅保存固定大小的 state/output/proof/DA-root 字段，并在争议场景中依赖 DA retrieval 获取详细 replay evidence。

| 方案             | 链上字节数                                      | 是否可重放                     | 证据获取方式                        | 验证成本                           |
|----------------|--------------------------------------------|---------------------------|-------------------------------|--------------------------------|
| full payload   | payload=8192、prompt=64 时平均 50,068.84 bytes | 是，payload 直接存在            | 无需 DA fetch                   | 链上字节开销高                        |
| compact commit | 406.00 bytes                               | 是，前提是 DA entry 与 proof 可用 | DA pointer + 字段级 Merkle proof | DA fetch + Merkle verification |

表 7 full payload 与 compact commit 的对比

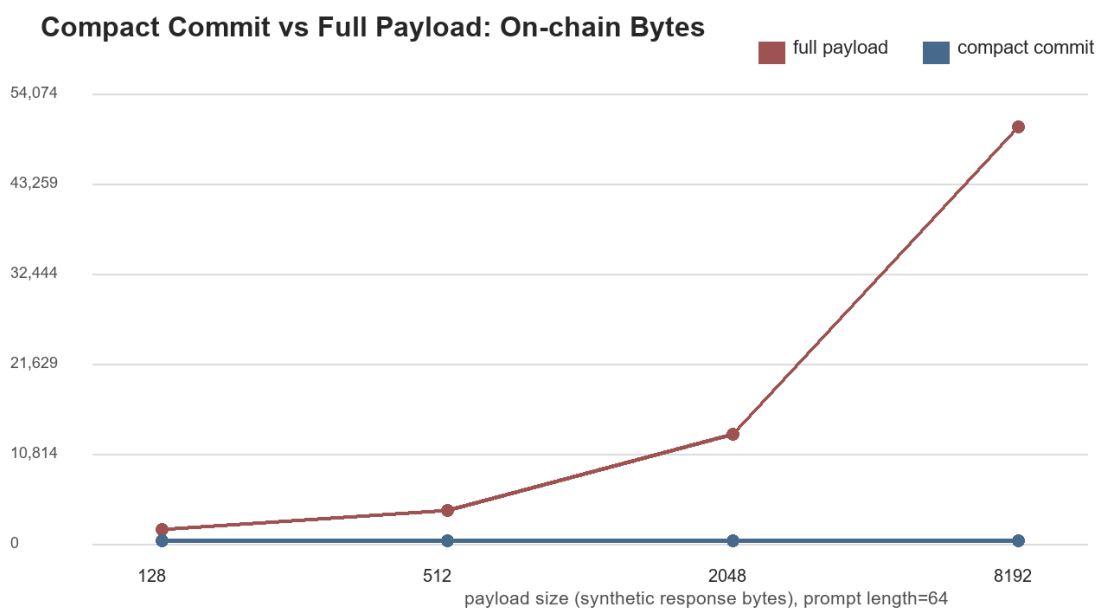


图 11 full payload 随合成 payload size 增长，而 compact commit 在原型编码中保持固定规模

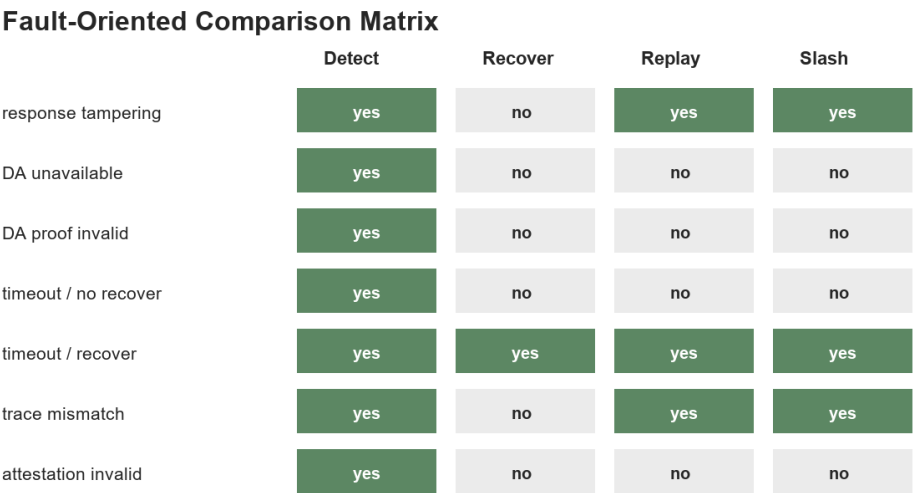
结果说明，该对比关注的不只是压缩。普通 hash 也可以降低字节数，但无法说明如何取回争议 response、验证字段级 membership 或重放相关证据。compact commit 的意义在于，在保持正常路径轻量的同时，为异常路径保留证据入口。其权衡也很明确：争议场景需要承担 DA fetch 与 Merkle verification 开销，但正常交互不必完整发布 payload 到链上。

7.7.3 面向故障分类的对比分析

安全评估不应将所有故障压缩为单一的成功/失败结果。下表区分每类注入故障在当前原型中是否可检测、可恢复、可重放和可惩罚。该表反映的是原型能力边界，而不是生产系统的通用故障处理结论。

| 故障类型                 | 可检测 | 可恢复 | 可重放 | 可惩罚 |
|----------------------|-----|-----|-----|-----|
| response tampering   | 是   | 否   | 是   | 是   |
| DA unavailable       | 是   | 否   | 否   | 否   |
| DA proof invalid     | 是   | 否   | 否   | 否   |
| timeout / no recover | 是   | 否   | 否   | 否   |
| timeout / recover    | 是   | 是   | 是   | 是   |
| trace mismatch       | 是   | 否   | 是   | 是   |
| attestation invalid  | 是   | 否   | 否   | 否   |

表 8 面向故障分类的检测、恢复、重放与惩罚对比



The matrix separates detectable faults from faults the current prototype can recover, replay, and slash.

图 12 面向故障分类的对比热力图

对比结果表明，response tampering 和 trace mismatch 可以进入 replay 与 slashing，因为原型能够构造可比较的 expected trace 与 claimed trace。DA unavailable、DA proof invalid 和 attestation invalid 可以被检测，但尚未被统一映射到 slashing 路径。该区分很重要：若

将所有可检测故障都写成可惩罚故障，将会夸大当前系统能力。故障分类视角能够更清楚地说明协议闭环已经覆盖哪些场景，哪些场景仍停留在检测或拒绝边界。

7.7.4 DA 路径权衡对比

DA 路径对比使用现有配置化 profile: full\_onchain\_calldata、compact\_external\_da、compact\_eip4844\_like 和 compact\_modular\_da\_sampling。图中固定 batch size=1000，以观察高频场景下的推销行为，并改变 payload size。所有数值均为脚本配置得到的 gas-equivalent estimate，不是实时网络价格。

| DA 路径                       | 推销成本作用                                | 重放准备性                     | payload 扩展性                      | DA 依赖            |
|-----------------------------|---------------------------------------|---------------------------|----------------------------------|------------------|
| full_onchain_calldata       | 大 payload 下配置化成本最高                    | payload 直接可用              | 链上字节线性增长                         | 无外部 DA 依赖        |
| compact_external_da         | 链上提交更轻，但 DA profile 成本高于其他 compact 路径 | 需要 DA proof               | payload 成本转移到 DA profile         | 依赖外部 DA 可用性      |
| compact_eip4844_like        | 配置化边际数据成本较低                           | 需要类 blob 数据获取             | 随 payload 与 batch 增大而更有利         | 依赖类 blob 数据可用性假设 |
| compact_modular_da_sampling | 配置化边际数据成本最低                           | 需要 DA sampling/profile 支撑 | 在高 payload、高 batch workload 下最有利 | 对独立 DA 层依赖最强     |

表 9 DA 路径权衡对比

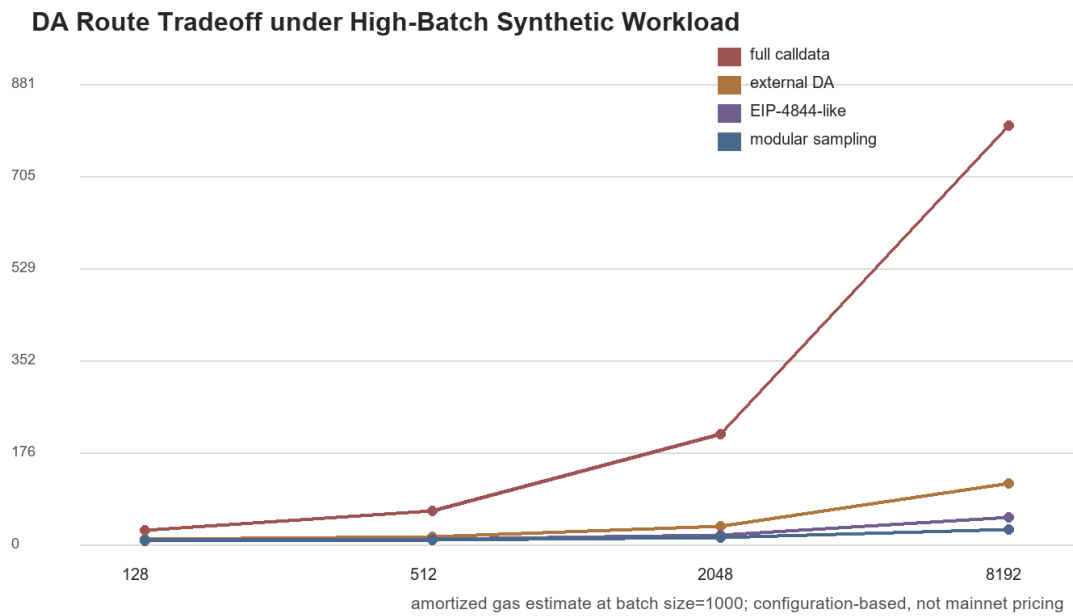


图 13 高 batch 合成负载下的 DA 路径权衡

可以观察到, full calldata 在 payload 增长时劣化明显, 因为完整 payload bytes 始终面向链上提交。compact 路线将链上提交与 payload size 解耦, 因此在高 payload、高 batch workload 下相对优势更明显。compact modular DA sampling 在该配置化估算中最有利, 是因为其边际 DA 成本最低; 但这也意味着系统更依赖独立 DA 层。该对比支持的是路径选择中的权衡判断, 而不是生产成本排序。

### 7.7.5 可扩展性边界

当前原型尚未产生并发挑战 benchmark。实验输出中没有 multiple concurrent challenge sessions、recovery queue growth、replay queue delay 或 scheduler contention 等测量。因此, 本文不增加生产级并发可扩展性主张。现有 challenge\_grid 仅支持说明 trace length 下的对数二分趋势; 并发恢复与重放调度仍属于后续工作。

## 8 讨论与局限性 (Discussion and Limitations)

本节将局限性与论文主张一并讨论。本文的价值不在于给出生产就绪的 Rollup, 而在于在限定假设下形成一个安全系统论证: recoverable challenge 改善仲裁连续性, evidence-carrying compact commit 保留 replay 上下文, DA-aware estimation 刻画高频 workload 下的成本边界。模拟 TEE、mock/verifiable DA registry、合成负载和本地 EVM baseline 限制了外部效应, 但并不改变本文关注的机制层问题。

### 8.1 内部效度 (Internal Validity)

内部效度主要受 simulated TEE、mock/verifiable DA registry、Python 原型逻辑与 Hardhat 本地 EVM 测量方式影响。原型中的 attestation 只建模输入/输出绑定, DA 层也没有接入真实采样与经济安全机制, 因此实验验证的是机制接口是否能跑通, 而不是硬件安全或 DA 经济安全。本地 gasUsed 还可能受到合约简化、测试数据规模固定和本地链环境的影响。

### 8.2 外部效度 (External Validity)

外部效度的限制来自实验环境到主网或生产网络之间的差距。主网 MEV、拥堵费率波动、blob 价格变化、真实 TEE 供应链攻击、侧信道风险、宿主 I/O 操控与真实 DA 网络运营风险, 均未在当前实验中覆盖。Sepolia 部署只能说明核心合约具有公开测试网可部署性, 不能据此外推主网性能或安全强度。

### 8.3 构念效度与部署边界 (Construct Validity and Deployment Boundary)

构念效度关注指标与论文主张之间是否匹配。challenge success、recovery\_success 与 slashed\_rate 衡量的是原型状态机下的仲裁推进, 而不是完整 Rollup 经济安全; DA-aware cost model 是配置化趋势模型, 不等价于真实 calldata/blob 定价; fault taxonomy 也尚未形成统一 slashing 闭环。因此, 论文结论应理解为机制层研究原型结论, 而非生产系统审计结果。

具体而言，TEE 部分仍停留在模拟层。attestation 用于绑定输入、输出、nonce 和执行环境标识，目的在于验证“链下执行摘要 - 链上提交 - 异常挑战”的协议接口；真实 SGX、TDX、CSV 或其他硬件 TEE 还需要处理远程证明格式、硬件漏洞、侧信道攻击、宿主 I/O 操控和多厂商信任假设。

DA 部分同样是 mock/verifiable DA 原型。系统实现了 DA root、payload hash、字段级 Merkle proof 和可用性状态记录，但没有接入 Celestia、EigenDA 或其他真实 DA 网络。因而，compact commit 与 verifiable DA structure 在本文中只能说明可检查证据入口的形成方式，不能等同于生产级 DA availability。

成本模型也应按配置化估算理解。DA profile 中的 gas per byte 参数用于构造可比较的趋势模型，而不是主网 calldata、blob 或真实 DA 服务的实时价格。由此得到的 96.33% 降本只对应给定 cost model 下的趋势结果，不能外推为主网级或生产级费用结论。

Hardhat gasUsed 提供的是本地 EVM 基线。它说明 Solidity 原型中的 submit、challenge open、recover、replay、resolve 和 finalize 等关键路径具有链上状态推进对应物，并能被本地测试链测量；但它不等价于 Sepolia 或主网费用，也没有覆盖公网拥堵、交易排序、blob 价格和跨合约部署环境差异。

Sepolia 部署记录属于公开测试网 deployability evidence。部署区块、合约地址和 ABI 索引说明当前合约可以部署到公开测试网；同时，源码验证曾受 block explorer connect timeout 影响，也尚未记录大规模 Sepolia 交互交易样本。该证据不应被解释为生产成熟度、主网性能或完整安全性的证明。

fault taxonomy 仍需继续收敛。当前 response\_tampered、trace\_length\_mismatch 和 challenge\_timeout\_recover 可以进入 challenge + slashing 路径；attestation\_invalid、da\_unavailable 和 da\_proof\_invalid 主要体现为检测、拒绝或报告能力。后续需要明确不同 fault 的仲裁策略，例如哪些故障应直接拒绝，哪些故障需要 watchdog 恢复，哪些故障应进入链上仲裁或经济惩罚。

challenge replay 中的部分 evidence 仍由链下提供。原型通过 expected hash、claimed hash、trace evidence 和 mismatch bits 支持 replay arbitration，但没有实现完整链上可验证执行环境、FPVM 或真实模型推理单步证明。下一步需要加强 evidence 的链上可验证性、形式化安全论证和跨实现一致性测试，减少 replay 结果对链下诚实生成证据的依赖。

综合来看，这些边界限定了后续研究方向，而不是削弱本文的核心机制主张。后续工作需要用真实 TEE evidence 替代 simulated attestation，接入真实 DA 网络，测量并发 challenge 队列，加强 replay evidence 的链上可验证性，并形式化 fault-to-slashing 策略。当前贡献仍在于：在限定假设下，通过原型证据展示“故障发生—恢复—重放—仲裁结算”的协议路径。

## 8.4 与相关工作的关系（Relationship to Prior Work）

本文属于对现有机制的增强，而非重新设计完整系统。本文并不试图替代 TEERollup、OTR 或 opML 等系统路线，而是在其异常路径上补充 Recoverable Challenge 与 Evidence-Carrying DA Binding，使 Hybrid TEE-Rollup 在 timeout、DA 异常与轻量提交场景下具备更清晰的仲裁推进能力。

TEEROLLUP 提出了利用异构 TEE 降低 Rollup 验证成本并缩短提现延迟的系统设计，其核心包括 TEE 委员会、链上状态合约、挑战机制和数据可用性惩罚。本文受到 Hybrid TEE-Rollup 思路启发，但并不重做完整 TEEROLLUP 系统，而是聚焦异常场景下 challenge 的恢复性，以及 compact commit 与 verifiable DA 的成本和证据结构。换言之，本文的贡献位置不是替代既有 TEE-Rollup 架构，而是在其“TEE 快速路径 + 链上异常路径”的思想下，补充 timeout 后争议会话如何恢复推进、轻量提交如何保留 DA 证据入口、以及成本趋势如何以可复现实验方式表达。

OTR 和 Optimistic TEE-Rollups 关注如何将 TEE 与 optimistic verification 结合，尤其适用于链上生成式 AI 或模型推理结果验证。这类工作表明 TEE 可以作为快速路径，而 optimistic challenge 可作为异常路径。本文与其相近之处在于同样关注 TEE 与 challenge 的结合，差异在于本文强调 recoverable challenge 的活性语义，以及 DA payload 与 compact commit 之间的证据绑定。

opML 等工作研究如何通过 optimistic fraud proof 支持机器学习或大模型计算，通常包含二分定位、单步仲裁和链上虚拟机验证。本文借鉴交互式争议定位思想，但没有实现完整 FPVM 或真实 ML 执行证明，而是将 bisection replay 用于 Hybrid TEE-Rollup 原型中的异常仲裁。

Dynamic Fraud Proof 关注如何缩短无争议场景下的最终性，并在发现争议时动态延迟结算。该方向强调 challenge window 和验证者参与机制的动态调整。本文不提出完整动态最终性协议，而是在更小范围内研究 timeout 后 challenge session 如何恢复推进。

LazyLedger 和 Light Clients for Lazy Blockchains 等工作强调将数据可用性从执行验证中解耦，并通过数据可用性抽样支持模块化扩容。本文没有实现真实 DAS 网络，而是使用 mock DA 和 DA profile 建模 compact commit + verifiable DA 的成本与证据结构。本文重点不是替代 DA 层，而是说明 Hybrid TEE-Rollup 中链上轻量提交必须与 DA 证明建立可验证联系。

EIP-4844 将 blob-carrying transaction 引入以太坊扩容路线，为 Rollup 数据发布提供了区别于传统 calldata 的成本结构。本文中的 compact EIP-4844-like profile 只抽象了“数据发布成本低于 full calldata”的趋势，并未模拟真实 blob base fee、blob 生命周期或主网交易调度。因此，EIP-4844 在本文中是成本建模参照，而不是已经接入的真实数据通道。

表 10 相关工作与本文工作的对照

| 方向 / 工作                       | 核心关注                                   | 与本文关系                                                 | 本文不声称覆盖的部分                   |
|-------------------------------|----------------------------------------|-------------------------------------------------------|------------------------------|
| TEEROLLUP / Hybrid TEE-Rollup | 利用异构 TEE 降低 Rollup 验证成本，并保留挑战和 DA 处罚机制 | 本文沿用 Hybrid TEE-Rollup 的研究方向，聚焦 recoverable challenge | 不实现完整 TEE 委员会、生产级提现流程或完整经济安全 |

| 方向 / 工作                            | 核心关注                                   | 与本文关系                                                            | 本文不声称覆盖的部分                          |
|------------------------------------|----------------------------------------|------------------------------------------------------------------|-------------------------------------|
|                                    |                                        | 与 evidence-carrying compact commit                               |                                     |
| Optimistic TEE-Rollups / OTR       | 将 TEE 快速执行与 optimistic verification 结合 | 本文同样采用“正常路径低成本、异常路径挑战”的思想                                        | 不证明真实 TEE 远程证明安全或链上生成式 AI 生产服务可用性   |
| opML / optimistic ML fraud proof   | 通过二分定位和单步证明验证 ML 或复杂计算                 | 本文借鉴 bisection replay 的争议定位结构                                    | 不实现完整 FPVM、真实模型推理单步证明或链上 ML 执行环境    |
| Dynamic Fraud Proof                | 动态调整争议窗口和最终性，以缩短无争议路径延迟                | 本文关注 timeout 后会话恢复，是更窄的 challenge liveness 问题                    | 不提出完整动态最终性协议                        |
| LazyLedger / 模块化 DA                | 将 DA 与执行验证解耦，支持数据可用性抽样                 | 本文用 mock/verifiable DA 与 DA profile 说明 compact commit 需要绑定 DA 证据 | 不接入真实 DA 网络，也不证明 DAS 经济安全           |
| Light Clients for Lazy Blockchains | 面向轻客户端的数据可用性抽样与验证                      | 本文借鉴“轻量验证者需要可检查数据承诺”的思想                                          | 不实现真实轻客户端协议                         |
| EIP-4844 / blob data               | 降低 Rollup 数据发布成本，提供 blob 数据市场          | 本文将 EIP-4844-like 作为配置化成本 profile                                | 不模拟真实 blob base fee、主网拥堵或 blob 生命周期 |

从对照可以看出，已有工作已经提供 TEE-Rollup、optimistic fraud proof 和 modular DA 等主要路线；本文将 recoverable challenge、DA-bound compact commit 和轻量成本模型放入同一个可运行研究原型中，并分别用 Python、Hardhat 与 Sepolia 证据说明机制可行性、链上对应物和可部署性边界。

## 8.5 小结与未来方向（Takeaways and Future Direction）

高频 DApp 的核心困难不只是吞吐量不足，还在于低成本交互与异常可验证之间经常互相拉扯。完全上链会把大量低价值高频请求转化为高昂链上成本；完全链下或单纯依赖 TEE 又会削弱异常时的公开仲裁能力。本文在已有 Hybrid TEE-Rollup 思路下回答的是一个更窄的问题：能否在不重做完整 Rollup 系统的前提下，让轻量提交、DA 证据和可恢复 challenge 形成可运行的研究原型。

现有证据给出了谨慎但有意义的回答。Python 原型跑通了 commit、DA proof、challenge、recovery、replay 与 resolution；对比型实验区分了检测与完成、压缩与证据绑定；本地 EVM 和 Sepolia 证据说明协议路径具有合约对应物。这些证据足以支撑限定假设下的机制设计主张，但不足以宣称生产级安全性或性能。

后续研究可以沿两个方向推进：一是把 evidence 的链上可验证性做得更强，减少 replay 对链下证据生成的依赖；二是在真实 DA 与 TEE 环境中重新测量成本、活性和异常处理边界。这样，Hybrid TEE-Rollup 面向高频 DApp 的可验证低成本交互，才可能从研究原型继续走向更完整的系统评估。

### 8.5.1 理论启示 (Theoretical Implication)

从理论层面看，本文更有价值的部分不是提出新的 Rollup 系统，而是将 Challenge Recovery 从工程补丁提升为具有协议语义的独立机制。Recoverable Challenge、Evidence-Carrying Commit 与 DA-Aware Cost Model 的组合，为 Hybrid TEE-Rollup 在高频 DApp 场景下的异常处理与低成本验证提供了一个可讨论的研究视角。

## 参考文献

- [8] Intel Corporation. Intel Software Guard Extensions (Intel SGX) Developer Guide[EB/OL]. 2024.
- [9] Intel Corporation. Intel Trust Domain Extensions (Intel TDX) Architecture Specification[EB/OL]. 2024.
- [10] Offchain Labs. Arbitrum: A Next-Generation Layer 2 for Ethereum[EB/OL]. 2021.
- [11] Cartesi. Cartesi Rollups Documentation[EB/OL]. 2024.
- [12] EigenLayer. EigenDA Documentation[EB/OL]. 2024.
- [13] Celestia. Celestia: Modular Blockchain Network[EB/OL]. 2024.
- [14] RISC Zero. RISC Zero zkVM Documentation[EB/OL]. 2024.
- [15] Succinct Labs. SP1: A Performant, 100% Open-Source zkVM[EB/OL]. 2024.
- [16] Arbitrum Foundation. Arbitrum Nitro Technical Documentation[EB/OL]. 2024.
- [17] OP Labs. Optimism Documentation: Fault Proofs and Dispute Games[EB/OL]. 2024.
- [18] Canetti R, et al. Universally Composable Security: A New Paradigm for Cryptographic Protocols[EB/OL]. 2000.
- [19] Goldwasser S, Micali S, Rackoff R. The Knowledge Complexity of Interactive Proof Systems[J]. SIAM Journal on Computing, 1989, 18(1): 186-208.
- [20] NIST. Recommendation for Key Management: Part 1 - General (Rev. 5)[EB/OL]. 2020.
- [1] WEN X, FENG Q, LYU H, et al. TEEROLLUP: Efficient Rollup Design Using Heterogeneous TEE[EB/OL]. arXiv:2409.14647, 2024.
- [2] PICCO G, FORTUGNO A. Dynamic Fraud Proof[EB/OL]. arXiv:2502.10321, 2025.
- [3] AL-BASSAM M. LazyLedger: A Distributed Data Availability Ledger with Client-Side Smart Contracts[EB/OL]. arXiv:1905.09274, 2019.
- [4] TAS E N, TSE D, YANG L, et al. Light Clients for Lazy Blockchains[EB/OL]. arXiv:2203.15968, 2022.
- [5] Ethereum Foundation. EIP-4844: Shard Blob Transactions[EB/OL]. 2024.
- [6] Ethereum Foundation. Optimistic Rollups[EB/OL]. Ethereum Documentation, 2024.
- [7] Buterin V. An Incomplete Guide to Rollups[EB/OL]. 2021.